



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Exception Handling, File I/O, Modules and Packages

Python Modules and Packages

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

Modules

- If a single Python file is created to perform some tasks, that's called a script.
- If a Python file is created to store functions, classes, and other definitions, that's called a module.
- A module is a file that contains Python code ending with the .py extension.



PROGRAMMING WITH PYTHON

Modules

- Modules are simply Python files, nothing more.
- The single file in a small program is a module.
- Two Python files are two modules.
- If there are two files in the same folder, a class from one module can be loaded for use in the other module.



PROGRAMMING WITH PYTHON

Modules

- Modules are files with “.py” extension containing Python code.
- They help to organize related functions, classes or any code block in the same file.
- It is considered as a best practice to split the large Python code blocks into modules containing up to 300–400 lines of code.



PROGRAMMING WITH PYTHON

Reason for Modules

- They allow code organization.
 - Example
 - Store all functions and data related to math in a module called `math.py`.



PROGRAMMING WITH PYTHON

Reason for Modules

- Reuse code more easily.
 - Instead of copying the same functions from one project to the next, use the module instead.
 - If the module is packaged properly, it can be shared with others on the Python package index.



PROGRAMMING WITH PYTHON

Modules

- The import statement is used for importing modules or specific classes or functions from modules.
- Example
 - A module called database.py, which contains a class called Database.
 - A second module called products.py is responsible for product-related queries.



PROGRAMMING WITH PYTHON

Modules

- `products.py` needs to instantiate the `Database` class from `database.py` so that it can execute queries on the product table in the database.
- There are several variations on the import statement syntax that can be used to access the class



PROGRAMMING WITH PYTHON

Modules

- ```
import database
db = database.Database()
Do queries on db
```
- This version imports the database module into the products namespace (the list of names currently accessible in a module or function), so any class or function in the database module can be accessed using the database.<something> notation.



# PROGRAMMING WITH PYTHON

## Modules

- ```
from database import Database  
db = Database()  
# Do queries on db
```
- import just the one class needed using the **from...import** syntax



PROGRAMMING WITH PYTHON

Modules

- `from database import Database as DB`
`db = DB()`
- *# Do queries on db*

- If, for some reason, products already has a class called Database, and don't want the two names to be confused, rename the class when used inside the products module



PROGRAMMING WITH PYTHON

Modules

- `from database import Database, Query`

- If the database module also contains a Query class, import both classes.

```
from database import *
```

- Import all classes and functions from the database module



PROGRAMMING WITH PYTHON

Organizing Modules

- A package is a collection of modules in a folder.
- Packages group similar modules in a separate directory.
- They are folders containing related modules and an `__init__.py` file which is used for optional package-level initialization.
- The name of the package is the name of the folder.



PROGRAMMING WITH PYTHON

Organizing Modules

- Tell Python that a folder is a package to distinguish it from other folders in the directory.
- To do this, place a (normally empty) file in the folder named `__init__.py`.



PROGRAMMING WITH PYTHON

Organizing Modules

- Retail store management system
 - Inventory management: Functions for adding and removing products.
 - Billing: Functions for calculating bills and applying discounts.
 - Customer management: Functions to manage customer details.
 - Utilities: General-purpose functions like logging and date formatting.



PROGRAMMING WITH PYTHON

Organizing Modules

- To organize the project
 - Use modules to separate related functionalities into individual Python files.
 - Group these modules into packages for better organization and reusability.



PROGRAMMING WITH PYTHON

Organizing Modules

- retail_management/
 - ├─ __init__.py
 - ├─ inventory/
 - │ └─ __init__.py
 - │ └─ add_product.py
 - │ └─ remove_product.py
 - ├─ billing/
 - │ └─ __init__.py
 - │ └─ calculate_bill.py
 - │ └─ discounts.py
 - ├─ customers/
 - │ └─ __init__.py
 - │ └─ add_customer.py
 - │ └─ remove_customer.py
 - ├─ utilities/
 - │ └─ __init__.py
 - │ └─ logger.py
 - │ └─ date_formatter.py

📁 / Retail Management /



Name



📁 billing



📁 inventory



📁 utilities



main.py



PROGRAMMING WITH PYTHON

Organizing Modules

- ```
def add_product(product_name, quantity):
 print(f"Product '{product_name}' with quantity {quantity} added to inventory.")
```

add\_product.py

```
1 def calculate_bill(cart):
2 total = sum(item['price'] * item['quantity'] for item in cart)
3 return total
```

calculate\_bill.py

```
1 def log_message(message):
2 print(f"LOG: {message}")
```

logger.py



# PROGRAMMING WITH PYTHON

## Organizing Modules

main.py

```
1 # Importing functions from various modules
2 from inventory.add_product import add_product
3 from billing.calculate_bill import calculate_bill
4 from utilities.logger import log_message
5 log_message("Application started")
6 add_product("Laptop", 10)
7 cart = [
8 {'name': 'Laptop', 'price': 100000, 'quantity': 2},
9 {'name': 'Mouse', 'price': 250, 'quantity': 5}
10]
11 total = calculate_bill(cart)
12 log_message(f"The total bill is: Rs.{total}")
```



# PROGRAMMING WITH PYTHON

## Organizing Modules

```
PS E:\MCA\Notes_MCA\FirstYear\1Sem\Computer Applications\Python\1Sem_2024\Programs\Unit3\Retail Management> python main.py
```

- LOG: Application started

Product 'Laptop' with quantity 10 added to inventory.

LOG: The total bill is: Rs.201250



# PROGRAMMING WITH PYTHON

## Organizing Modules

---

- There are two ways of importing modules
- absolute imports
- relative imports



# PROGRAMMING WITH PYTHON

## Organizing Modules – Absolute Import

---

- This method specifies the complete path to the module, function, or class needed to be imported.
- Example
  - It is the most common and clear way to import modules in Python, as it avoids ambiguity and works regardless of the current working directory.



# PROGRAMMING WITH PYTHON

## Organizing Modules – Absolute Import

- School management system
  - Student Management: Adding, removing, and updating student records.
  - Teacher Management: Adding, removing, and updating teacher records.
  - Utilities: General-purpose functionality, such as logging and formatting data.





# PROGRAMMING WITH PYTHON

## Organizing Modules – Absolute Import

- school\_management/
  - ├─ main.py
  - ├─ student/
    - │ └─ \_\_init\_\_.py
    - │ └─ add\_student.py
    - │ └─ remove\_student.py
  - ├─ teacher/
    - │ └─ \_\_init\_\_.py
    - │ └─ add\_teacher.py
    - │ └─ remove\_teacher.py
  - └─ utilities/
    - │ └─ \_\_init\_\_.py
    - │ └─ logger.py
    - │ └─ formatter.py

|                                     |           |
|-------------------------------------|-----------|
| / Student Management /              |           |
| Name                                |           |
| <input type="checkbox"/>            | Student   |
| <input type="checkbox"/>            | Teacher   |
| <input type="checkbox"/>            | utilities |
| <input checked="" type="checkbox"/> | main.py   |



# PROGRAMMING WITH PYTHON

## Organizing Modules – Absolute Import

```
1 def add_student(name, grade):
2 print(f"Added student: {name}, Grade: {grade}")
```

add\_student.py

```
1 def add_teacher(name, subject):
2 print(f"Added teacher: {name}, Subject: {subject}")
```

add\_teacher.py

```
1 def log_message(message):
2 print(f"LOG: {message}")
```

logger.py



# PROGRAMMING WITH PYTHON

## Organizing Modules – Absolute Import

```
1 from Student.add_student import add_student
2 from Teacher.add_teacher import add_teacher
3 from utilities.logger import log_message
4
5 log_message("Application started")
6 add_student("Aarav", "10th Grade")
7 add_teacher("Mr. Rudra", "Mathematics")
8 log_message("Application ended")
```

Absolute Import

```
PS E:\MCA\Notes_MCA\FirstYear\1Sem\Computer Applications\Python\1Sem_2024\Programs\Unit3\Student Management> python main.py
LOG: Application started
Added student: Aarav, Grade: 10th Grade
Added teacher: Mr. Rudra, Subject: Mathematics
LOG: Application ended
```



# PROGRAMMING WITH PYTHON

## Organizing Modules - Absolute Import

---

- Absolute imports are preferred because they are quite clear and straightforward.
- It is easy to tell exactly where the imported resource is, just by looking at the statement.
- Additionally, absolute imports remain valid even if the current location of the import statement changes.



# PROGRAMMING WITH PYTHON

## Organizing Modules – Relative Import

- When working with related modules inside a package, it seems kind of redundant to specify the full path.
- Relative imports are basically a way of saying find a class, function, or module as it is positioned relative to the current module.
- Example, if working in the products module and there is a need import the Database class from the database module next to it, use a relative import.



# PROGRAMMING WITH PYTHON

## Organizing Modules – Relative Import

- Library management system
  - Books: Manage adding, removing, and searching books.
  - Members: Manage member registrations and renewals.
  - Utilities: General-purpose helpers like logging and formatting.



# PROGRAMMING WITH PYTHON

## Organizing Modules – Relative Import

```
library_management/
├─ main.py
├─ books/
│ ├─ __init__.py
│ ├─ add_book.py
│ ├─ remove_book.py
│ └─ search_book.py
├─ members/
│ ├─ __init__.py
│ ├─ add_member.py
│ └─ remove_member.py
├─ utilities/
│ ├─ __init__.py
│ ├─ logger.py
│ └─ formatter.py
```



# PROGRAMMING WITH PYTHON

## Organizing Modules – Relative Import

📁 / Library / library\_management /

☐ Name

☐ 📁 books

☐ 📁 members

☐ 📁 utilities

☐ 🐍 \_\_init\_\_.py

☐ 🐍 main.py

```
2 from ..utilities.logger import log_message
3
4 def add_book(title, author):
5 log_message(f"Adding book: {title} by {author}")
6 print(f"Book '{title}' by {author} added to the library.")
```

```
1 from ..utilities.formatter import format_name
2
3 def add_member(first_name, last_name):
4 full_name = format_name(first_name, last_name)
5 print(f"Member '{full_name}' added to the library.")
```

```
1 def format_name(first_name, last_name):
2 return f"{first_name} {last_name}"
```





# PROGRAMMING WITH PYTHON

## Organizing Modules – Relative Import

```
1 import sys
2 sys.path.append(".")
3 from books.add_book import add_book
4 from members.add_members import add_member
5
6 add_book("Pride and Prejudice", "Jane Austen")
7 add_member("Harry", "Potter")
```



# PROGRAMMING WITH PYTHON

## Organizing Modules – Relative Import

---

- Can use more periods to go further up the hierarchy.
- Can also go down one side and back up the other.



# PROGRAMMING WITH PYTHON

## Organizing Modules – Pros

| Absolute Imports                                                                                                                   | Relative Imports                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Absolute imports are preferred because they are quite clear and straightforward.</li></ul> | They are quite succinct.                                                                                       |
| It is easy to tell exactly where the imported resource is, just by looking at the statement.                                       | Depending on the current location, they can turn the ridiculously long import statement to something as simple |



# PROGRAMMING WITH PYTHON

## Organizing Modules – Pros

| Absolute Imports                                                                                                                                            | Relative Imports |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <ul style="list-style-type: none"><li>• Additionally, absolute imports remain valid even if the current location of the import statement changes.</li></ul> |                  |



# PROGRAMMING WITH PYTHON

## Organizing Modules – Cons

| Absolute Imports                                                                                                                                  | Relative Imports                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Absolute imports can get quite verbose, depending on the complexity of the directory structure.</li></ul> | Relative imports can be messy, particularly for shared projects where directory structure is likely to change.                |
|                                                                                                                                                   | Relative imports are also not as readable as absolute ones, and it's not easy to tell the location of the imported resources. |



# PROGRAMMING WITH PYTHON

## Recap

---

- Modules
- Organizing Modules
- Absolute Imports
- Relative Imports
- Pros and Cons



**THANK YOU**

---

**Lekha A**

Computer Applications